Ch 06: Processing File Records, Python Exceptions

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



Processing Records

- **Record**: set of data that describes one item
  - record, database row, structure, object, item, user-defined type (UDT), compound data type
- **Field**: single piece of data within a record
- Write record to sequential access file by writing the fields one after the other
- Read record from sequential access file by reading each field until record is complete

Records

The data that is stored in a file is frequently organized in records. A record is a complete set of data about an item, and a field is an individual piece of data within a record.

A Record (Wikipedia, 2026) is a basic computer science concept. Called user-defined type (UDT), compound data type, etc. Anytime you collect multiple fields together to form a compound set of data, you have a user-defined type in a data processing system.



EAST TEXAS A&M

Records

- A record is made up of multiple fields (name, id number, department)

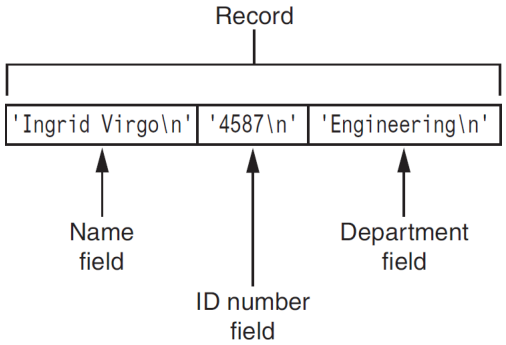


Figure 1: Fields in a record (Gaddis, 2021, pg.351)

EAST TEXAS A&M

Records

- Each time you write a record to a sequential access file, you write the fields that make up the record, one after the other

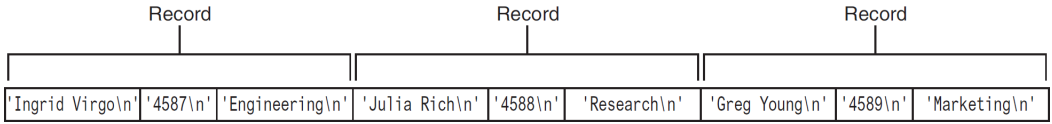


Figure 2: Records in a file (Gaddis, 2021, pg.351)

EAST TEXAS A&M

Application to Process Records

- When working with records (like a database), typically the following operations all need to be implemented
  - Add a record
  - Display records (generate reports)
  - Search for a specific record
  - Modify / update a record
  - Delete a record



EAST TEXAS A&M

Exceptions (1 of 3)

- **Exception**: error that occurs while a program is running
  - Usually causes program to abruptly halt
  - Can sometimes continue from an exception, sometimes cannot
- **Traceback**: error message that gives information regarding line numbers that caused the exception
  - Indicates the type of exception and brief description of the error that caused exception to be raised

Exceptions

An exception is an error that occurs while a program is running, causing the program to abruptly halt. You can use the `try / except` statement in Python to gracefully handle exceptions when anticipated or expected in your code.



Exceptions (2 of 3)

- Some (many?) exceptions can be prevented by careful coding
  - Example: input validation
  - Usually involve a simple decision construct
- Some exceptions cannot be avoided by careful coding
  - Examples
    - Trying to convert non-numeric string to an integer
    - Trying to open for reading a file that doesn't exist

EAST TEXAS A&M

Exceptions (3 of 3)

- **Exception handler**: code that responds when exceptions are raised and prevents program from stopping
  - In Python, written as `try / except` statement
  - **Try block**: statements that can potentially raise a (particular type of) exception
  - **Handler**: statements contained in `except` block that run if exception is raised in the `try` block
  - If statement in try block raises exception
    - Handler immediately following `except` clause executes
    - Continue program after `try / except` statement
  - Other exceptions:
    - Program halts with traceback error message
  - If no exception is raised, handlers are skipped

```
try:
    statement
    statement
except exceptionName:
    statement
```



Handling Multiple Exceptions

- Often code in `try` block can throw more than one type of exception
  - Need to write `except` clause for each type of exception that needs to be handled
- An `except` clause that does not list a specific exception will handle any exception that is raised in the `try` block
  - Should always be last in a series of `except` clauses
  - Usually a **BAD** code smell, should usually not try and handle any and all problems, even if unexpected

EAST TEXAS A&M

Displaying an Exception's Default Error Message

- Exception object: object created in memory when an exception is thrown
  - Usually contains default error message pertaining to the exception
  - Can assign the exception object to a variable in an `except` clause
    - Example: `except ValueError as err:`
  - Can pass exception object variable to `print()` function to display the default error message

EAST TEXAS A&M

The `else` Clause

- `try / except` statement may include an optional `else` clause, which appears after all the `except` clauses
  - Aligned with `try` and `except` clauses
  - Syntax similar to `else` clause in decision structure
  - `Else suite`: block of statements executed after statement in `try` block, only if no exceptions were raised
    - If exception was raised, the `else` suite is skipped

EAST TEXAS A&M

The `finally` Clause

- `try / except` statement may include an optional `finally` clause, which appears after all the except clauses
  - Aligned with `try` and `except` clauses
  - Finally suite: block of statements after the `finally` clause
    - Executes whether an exception occurs or not
    - Purpose is to perform cleanup before exiting

EAST TEXAS A&M

Summary

- A **Record** is a set of data that describes one item
  - This is a classic CS concept, see User-Defined Type (Wikipedia, 2026)
  - A record is composed of **Field**s, single pieces of data that compose the record
- Using a sequential file, you have to read all records into memory to add and modify, and write back whole file when done.
- **Exception**s are errors that occur while a program is running
- In Python you can use `try / except` statements to try and gracefully exceptions raised by programs



EAST TEXAS A&M

References

Gaddis, T. (2021). *Starting out with Python* (5th). Pearson.

Wikipedia. (2026). *Record (computer science)*. Retrieved June 20, 2026, from https://en.wikipedia.org/wiki/Record_(computer_science)

EAST TEXAS A&M